

Comparative Analysis of Manual and AI-Generated Shaft Design Codes

Design of Machine Elements

Department of Mechanical and Aerospace Engineering
University of California, San Diego
MAE 190

William Harris

Introduction:

For this project, I undertook a comparative analysis between my manually developed MATLAB code for shaft design (ShaftDesign.m) and an implementation generated by Perplexity AI (Perplexity.m). Both codes were designed to automate the shaft design process using the DE-Gerber criterion, but each took distinctly different approaches. Through this comparison, I aimed to understand both the capabilities and limitations of AI-generated engineering code while validating my own implementation approaches.

Program Comparison

When examining both implementations side by side, several key differences emerge in how each program approaches the shaft design problem. In developing my code, I chose to implement a modular structure that separates core functionalities into distinct sections, organizing the code into clear segments for constant definitions, material properties, calculation procedures, and helper functions. The AI-generated code, while functional, takes a more linear approach where these elements are intermingled, making it potentially more difficult to modify or maintain in the future.

These structural differences extend to how each program handles materials and properties. I implemented a comprehensive ASTM steel database supporting both hot-rolled and cold-drawn options, organized in a clear matrix format for accurate material selection and property lookup. The AI-generated code opts for a simpler approach with basic material input options and manual strength value entry, limiting its flexibility in real-world applications.

The handling of iteration and convergence reveals perhaps the most significant differences between our implementations. My code employs multiple convergence criteria, monitoring both diameter changes and safety factor achievement while incorporating proportional safety factor adjustment to handle complex interdependencies. Testing both codes with identical input conditions showed interesting results - my code calculated a minimum shaft diameter of 1.5386 inches with a fatigue safety factor of 1.5046 (targeting 1.5000) and a yielding safety factor of 2.3555, taking 5 iterations to converge. The AI-generated code produced a required shaft diameter of 1.5002 inches with a fatigue safety factor of 1.5000 and a yielding safety factor of 3.0138, completing in 4 iterations.

AI Code Generation and Engineering Validation

My experience with generating the shaft design code through AI revealed several key insights about effective prompting and code validation. The initial prompt I used included specific requirements and referenced textbook information, which proved essential for getting useful results. I provided detailed input parameters, material requirements, and the DE-Gerber formula implementation details, which helped the AI understand the engineering context more clearly. This experience demonstrated that well-structured prompts with supporting documentation lead to better initial results.

However, I discovered that error handling and debugging with AI-generated code presents unique challenges. When errors occurred, feeding them back to the AI sometimes led to corrections, but in other cases, the AI seemed to get stuck in patterns of incorrect implementation. I found that starting fresh conversations without the problematic history often produced better results than trying to debug within the same conversation.

The reproducibility of AI-generated code presented some interesting patterns. Even with similar prompts across different conversations, I found that the AI would often produce code with similar structures and, notably, similar issues. When an error or limitation appeared in one generated version, similar prompts tended to reproduce the same problems. This made it particularly challenging to obtain a fully functional implementation just through prompt refinement. Starting fresh conversations sometimes helped avoid previous errors, but fundamental limitations in the AI's understanding of engineering principles often resurfaced in new forms. This suggests that while AI can consistently generate basic functional code, relying solely on prompt engineering may not be sufficient to produce robust engineering software that handles all edge cases and requirements properly.

Discussion

The comparison between these implementations has revealed interesting insights about both approaches to automated shaft design. The significant difference in yielding safety factors between my code (2.3555) and the AI-generated version (3.0138) exemplifies how varying approaches to calculation and convergence can affect final results. While the AI code achieved reasonable results for basic cases, its simpler approach to material properties and convergence handling may not provide the same level of reliability for complex engineering scenarios. The differences in implementation highlight the importance of thorough engineering knowledge in developing robust design software.

Conclusion:

This analysis has illuminated both the potential and current limitations of AI in engineering software development. While AI can generate functional code relatively quickly, the additional robustness and reliability I built into my implementation through careful consideration of engineering principles and edge cases proved valuable for professional applications. The process highlighted the importance of proper prompting and validation techniques when working with AI-generated code. Moving forward, AI tools show promise as aids in the development process, but they require careful guidance and thorough validation to ensure their output meets engineering standards. The role of the engineer remains crucial in developing and validating reliable design tools, regardless of whether AI assistance is utilized in the development process.